

UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE

Centro de Tecnologia

Departamento de Engenharia de Computação e Automação

DCA0212.1 - Circuitos Digitais

Docente: Dr. Samuel Xavier de Souza

Desenvolvimento de operação para estender a funcionalidade de um processador programável

1.0 Introdução

Vamos realizar o projeto do processador programável de seis instruções descrito no capítulo 8 do livro do Frank Vahid, seguindo a abordagem de Projeto RTL (Register-Transfer Level), conforme explorado no capítulo 5 do livro *Digital Design*, de Frank Vahid. O Projeto RTL nos permite descrever circuitos digitais em termos de transferências de dados e operações realizadas entre registradores, entradas e saídas, com uma representação comportamental de alto nível.

Inicialmente, revisamos conceitos fundamentais, como o design de circuitos combinacionais no capítulo 2, onde o comportamento desejado foi capturado através de tabelas-verdade, e o design de circuitos sequenciais no capítulo 3, onde introduzimos Máquinas de Estado Finito (FSM) para capturar comportamentos mais complexos. No capítulo 4, avançamos para o estudo de datapaths, o que nos permitiu implementar abstrações mais avançadas.

No Projeto RTL que estamos desenvolvendo, utilizaremos uma Máquina de Estado de Alto Nível (HLSM) para capturar o comportamento do processador. A HLSM nos permitirá representar entradas e saídas de múltiplos bits, operações aritméticas e memória local.

Além disso, faremos uma extensão no processador de seis instruções, adicionando uma sétima instrução chamada JLT (Jump Less Than). Essa nova instrução introduz um desvio condicional que verifica se o valor de um registrador é menor que o de outro e, se for, será realizado um salto de instruções. Portanto, nosso objetivo será implementar um processador programável de seis instruções com a adição da instrução JLT, expandindo sua capacidade de tomar decisões complexas em tempo de execução.

Importante: Para fins de objetividade e otimização de espaço, as imagens não mostradas aqui estarão no slide da apresentação.

2.Desenvolvimento

O processador digital de propósito geral com sete instruções deverá ser capaz de receber da memória um código em Assembly, interpretá-lo e utilizar suas operações lógicas e aritméticas para manipular dados. Para compor o funcionamento desse processador, foram utilizadas as seguintes instruções:

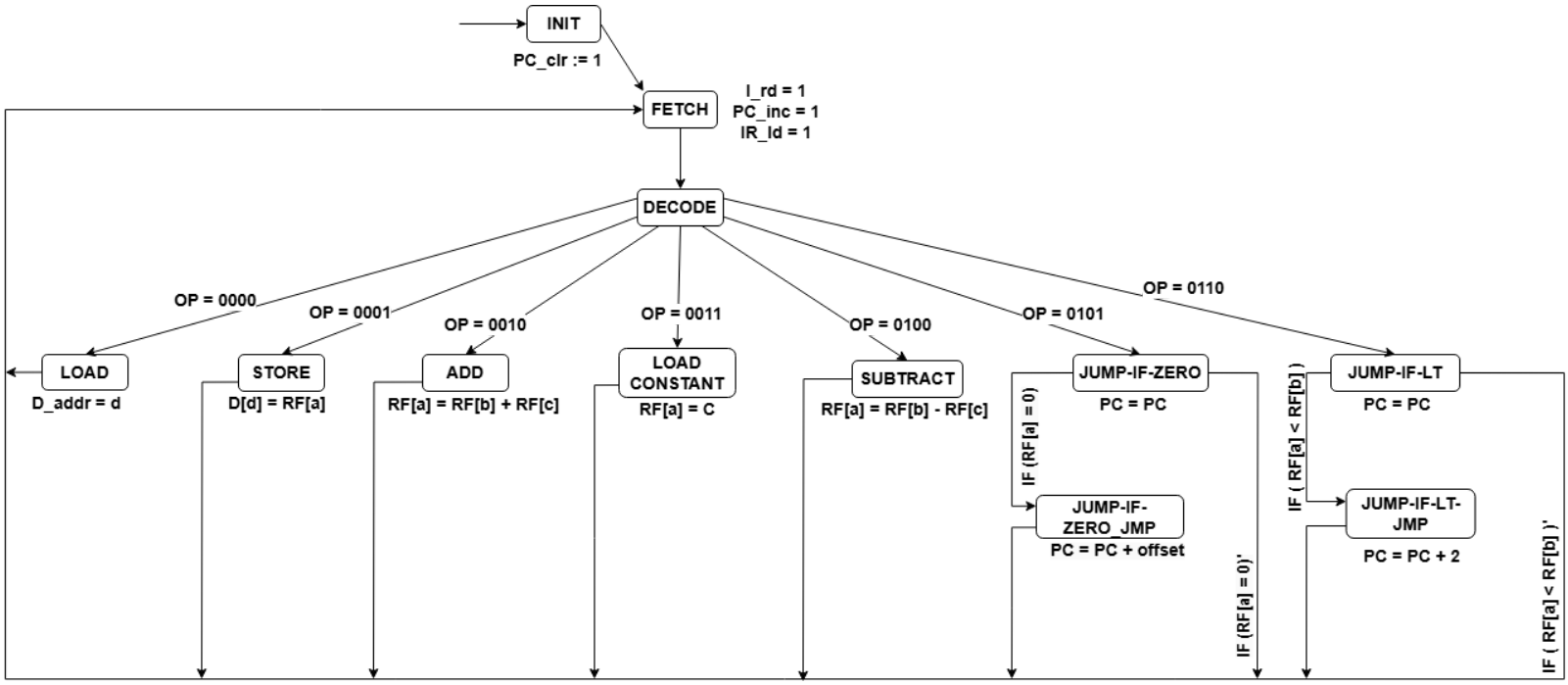
Instrução	Significado	Opcode
MOV Ra, d	RF[a] = D[d]	0000
MOV d, Ra	D[d] = RF[a]	0001
ADD Ra, Rb, Rc	RF[a] = RF[b] + RF[c]	0010
MOV Ra, #C	RF[a] = C	0011
SUB Ra, Rb, Rc	RF[a] = RF[b] - RF[c]	0100
JMPZ Ra, offset	PC = PC + offset if RF[a] = 0	0101
JLT Ra, Rb, Rc	PC = PC + 2, if RF[a] < RF[b]	0110

As instruções 'MOV Ra, d' e 'MOV d, Ra' transferem dados entre a memória e o banco de registradores, enquanto 'MOV Ra, #C' carrega um valor constante imediato C em um registrador. As operações aritméticas são realizadas pelas instruções 'ADD Ra, Rb, Rc' e 'SUB Ra, Rb, Rc', que somam e subtraem, respectivamente, os valores dos registradores Rb e Rc, armazenando o resultado em Ra. A instrução 'JMPZ Ra, offset' realiza um salto condicional para um novo endereço baseado no offset se o conteúdo de Ra for zero, e temos a nova

instrução 'JLT Ra, Rb, Rc' que “salta” instruções se R[a] for menor que R[b]. Estas instruções permitem ao processador executar uma variedade de operações essenciais para a execução de programas.

2.1. Projeto da máquina de estados de alto nível

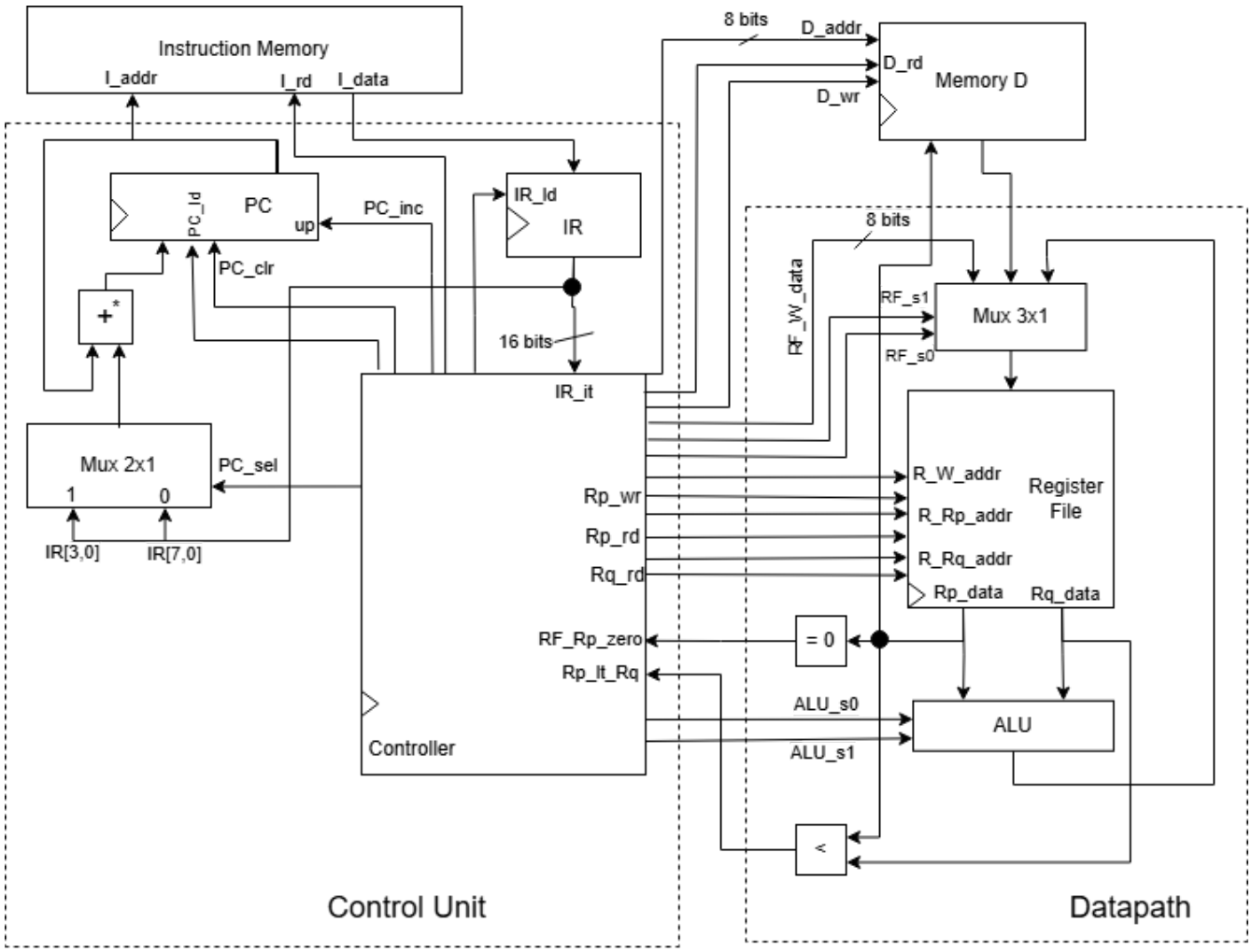
No diagrama da máquina de estados de alto nível temos as manipulações de dados no processador para cada operação onde RF[x] indica um dado do banco de registradores na posição X e D[x] indica um valor armazenado na posição X da memória.



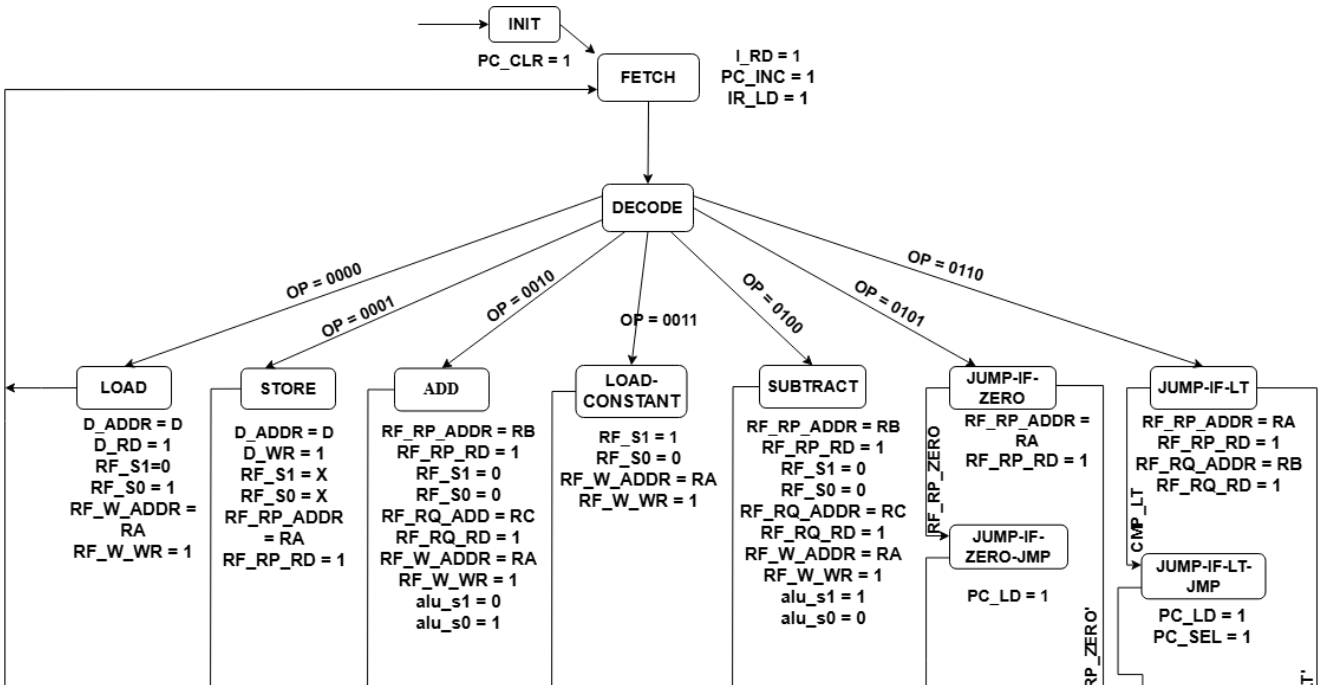
2.2. Projeto do caminho de dados e unidade de controle

Dada a representação do comportamento do processador programável a partir da máquina de estados de alto nível podemos derivar o caminho de dados para lidar com as necessidades do circuito digital. Tendo conhecimento que é necessário uma estrutura para lidar com os dados de cada operação, projetamos o caminho de dados do circuito que irá se comunicar com o controlador do processador.

O diagrama a seguir exibe a unidade de controle e suas conexões com datapath, a memória de instruções e a memória de dados D. O datapath é formado por um Mux3x1, um banco de registradores, uma unidade lógica aritmética e dois comparadores de magnitude. O datapath também possui ligações com a memória D.



2.3. Máquina de estados finitos do processador com as sete instruções



2.4. Código em VHDL para testar a implementação no Quartus

A implementação do processador foi feita no Quartus II usando como base os códigos disponibilizados na turma virtual. Para o teste, foi feito uma sequência de instruções utilizando várias instruções do processador, inclusive a instrução Jump Less Than como mostrado na imagem a seguir.

```
CONSTANT mem: rom_type :=(
    0 => "0011" & "0000" & "00000000",      -- CONST <- RF(0) = 0
    1 => "0011" & "0001" & "00000001",      -- CONST <- RF(1) = 1
    2 => "0011" & "0010" & "00000001",      -- CONST <- RF(2) = 0
    3 => "0011" & "0011" & "00001000",      -- CONST <- RF(3) = 8
    4 => "0001" & "0011" & "00000000",      -- STORE <- D(0)  = RF(3)
    5 => "0001" & "0010" & "00000000",      -- STORE <- D(0)  = RF(2)
    6 => "0010" & "0010" & "0010" & "0001", -- ADD   <- RF(2) = RF(2) + RF(1)
    7 => "0111" & "0011" & "0010" & "0010", -- JMPLT <- RF(3) < RF(2), JUMP 2
    8 => "0101" & "0000" & "11111101",      -- JMPZ  <- RF(0) = 0 JUMP -3
    9 => "0011" & "0010" & "11111111",      -- CONST <- RF(2) = 255
    10 => "0001" & "0010" & "00000000",     -- STORE <- D(0)  = RF(2)
    others => "0000000000000000" -- LOAD <- RF(0) = D(0);
);
```

O código utiliza quatro registradores para armazenar valores que serão utilizados em uma soma em que a condição de parada é determinada utilizando a instrução JMPLT. Como mostrado na instrução da linha 8, o valor armazenado em RF[3] será comparado com o valor armazenado em RF[2] e caso RF[3] seja menor haverá um salto para duas instruções a frente, ou seja, um incremento no valor do program counter (PC) baseado no valor passado nos último quatro bits do opcode do estado JMPLT. Caso a verificação não seja verdade a sequência de instruções segue normalmente e será feito um Jump If Zero ao verificar o valor de RF[0] que é zero e permanece zero intencionalmente. Esse Jump é feito para três instruções anteriores onde o valor da soma será armazenado (A operação store exibe o valor na simulação ou FPGA) na memória e a soma será feita novamente até que a verificação do JMPLT seja verdadeira e a próxima instrução será carregar a constante 255 no banco de registradores e então fazer o store deste valor. Ao fazer o armazenamento deste valor os oito Led's da FPGA que representam os bits serão acesos.

3.0 Dificuldades encontradas e soluções aplicadas

- Lidar com a quantidade de informações na HLSM e principalmente na FSM;
    - Usando o exemplo do slide do capítulo 8 foi possível fazer a adaptação para nosso trabalho e fazer a HLSM e FSM.
- Fazer o diagrama da unidade de controle e datapath com os componentes necessários para a extensão.
- Codificar o processador.
    - Usando o material de apoio disponibilizado na tarefa e o material do laboratório.

4.0 Conclusão

Neste projeto pudemos utilizar os conceitos aprendidos e as habilidades desenvolvidas durante a disciplina de Circuito Digitais, fazendo o uso de técnicas como utilizar a máquina de estados finitos de alto nível e a máquina de estados finitos para fazer a representação do funcionamento do processador de forma geral e mais abstrata, explicitando as ações que cada estado deveria fazer. Utilizamos a máquina de estados finitos de alto nível para desenvolver o diagrama do datapath de acordo com a sequência de desenvolvimento de um projeto RTL. Os componentes da unidade de controle também foram feitos junto com a adaptação do Datapath para suportar as sete instruções. O código do processador feito em VHDL foi feito com adição do material disponibilizado e códigos desenvolvidos nas aulas de laboratório. O código foi estruturado em módulos onde os componentes do circuito foram implementados e usados para construir componentes maiores. O resultado final do código foi um código capaz de receber linhas de instruções para realizar as sete operações.